

Part 6: Case Study

Summary

In this Part we will consider the **Zillow Prize**, a prediction challenge put forth on **Kaggle**, www.kaggle.com, a web site dedicated to data science challenges of this nature.

We will not pursue the full prediction challenge, but will tackle some of the issues faced when working with the data in its raw form, and explore how visualization can lead to some initial ideas on analyzing the data.

The Challenge

The challenge can be found at www.kaggle.com/c/zillow-prize-1.

Zillow's fundamental objective is to accurately predict the price at which houses will sell.

In this challenge, the company is hoping that data scientists can improve on their existing prediction algorithms. In particular, the **response** that they asking for predictions of is the **log error** in their current "Zestimate":

$$\text{response} = \log(\text{Zestimate}) - \log(\text{sale price})$$

Exercise: Why are they asking for estimates of the log error, instead of the actual price or the actual error?

The log Error is better than absolute error

For instance, an error of \$1000 is much more significant if value is \$100,000 than it is if it is \$1000,000,000

By focusing on the error, they are asking the data scientists to improve on their current algos, which have taken years to develop. They are looking for the next absolong improvement

Downloading the Data

On the "Data" for the challenge, there are three data sets that we require:

1. `properties_2016.csv` - The characteristics of all properties in Los Angeles, Orange, and Ventura counties in 2016.
2. `train_2016.csv` - The **training set** of all transactions from 1/1/2016 until 10/15/2016, plus some after 10/15/2016.
3. `zillow_data_dictionary.xlsx` - Descriptions of all of the variables in the properties file.

Download all three of these. (You will have to register for Kaggle.)

- 1 **Exercise:** What is the significance of the provided “training set” and how
- 2 does it differ from the “test set?”

3 The training set — the data used to fit the model, response is known for these

4 The test set — the response is not known, or is treated as being not known until the very end

5 _____

6 _____

7 _____

8 _____

9 _____

10 _____

11 _____

1 Understanding the Data Set

- 2 We should first understand the properties data set.
- 3 I assume that the file `properties_2016.csv` is in the current working
- 4 directory. A brief inspection of the file suggests (as the name implies) that
- 5 the entries in each row are separated by commas, and the first line of the file
- 6 contains the variable names.
- 7 Hence, we try the command

```
> prop2016 = read.table("properties_2016.csv",
+                       sep="," , header=T, na.strings="")
```

- 8 but we see that an error is produced. Unfortunately, this is not unusual.

- 1 **Exercise:** Resolve the problem and read in the data set.

2 Needed to include `comment.char=""` to override R's default of treating “#” as a comment

3 _____

4 _____

5 _____

6 _____

7 _____

8 _____

9 _____

10 _____

11 _____

- 1 This data set consists of 58 columns of data on each of 2985217 properties.
- 2 Take a look at

```
> names(prop2016)
```

- 3 to see the entire list of variables.
- 4 Our objective now is correctly characterize each of these variables as to
- 5 their type, and how, if at all, they could be used in an analysis that seeks
- 6 to predict the error in the Zestimate.

- 1 Going through the variables, I assign each to one of four groups: un-
- 2 ordered categorical, ordered categorical, continuous, or character.

```
> unorderedcat = c(2, 3, 7, 10, 18, 23, 24, 30, 31, 32, 33, 34,
+                 35, 36, 37, 38, 39, 40, 42, 44, 50, 56, 58)
> orderedcat = c(8)
> contvar = c(4, 5, 6, 9, 11, 12, 13, 14, 15, 16, 17, 19,
+            20, 21, 22, 25, 26, 27, 28, 29, 41, 43, 45,
+            46, 47, 48, 49, 51, 52, 53, 54, 55, 57)
> charvar = c(1)
```

- 3 There is some subjectivity in these choices. In particular, any variable with
- 4 only a few possible values could ultimately best be treated as a categorical
- 5 variable.

- 1 **Exercise:** Discuss the decisions made regarding each of the following vari-
- 2 ables: parcelid, latitude, longitude, and yearbuilt.

3 parcelid is a unique identifier for property

- 4 It does not make sense to treat it as factor, a factor is a variable that group obs in a meaningful way

5 Latitude and longitude treated as continuous

- 6 yearbuilt is treated as continuous. If treated as factor, could have a lot of levels

- 1 Now we will appropriately transform each variable:

```
> for(i in unorderedcat) prop2016[,i] =
+   factor(prop2016[,i])
> for(i in orderedcat) prop2016[,i] =
+   factor(prop2016[,i], ordered=TRUE)
> for(i in charvar) prop2016[,i] =
+   as.character(prop2016[,i])
```

- 2 and verify that the continuous columns are already in a numeric form:

```
> all(apply(prop2016[,contvar], 2, is.numeric))
[1] TRUE
```

- 1 **The Training Transactions**

- 2 Let's now read in the file of transactions in the training set:

```
> traintrans = read.table("train_2016_v2.csv",
+                          sep="," , header=T)
```

- 3 Transform the columns appropriately:

```
> traintrans$parcelid =
+   as.character(traintrans$parcelid)
> traintrans$transactiondate =
+   as.Date(traintrans$transactiondate,
+           format="%Y-%m-%d")
```

1 I want to **merge** these data frames in the following way: I want a new data
2 frame which has each transaction as a row, with the characteristics of the
3 property under consideration along with the sale details.

4 We want to use `parcelid` in order to achieve the matching. This will be
5 simplest if we can be comfortable assuming every parcel ID in `prop2016`
6 is unique:

```
> length(unique(prop2016$parcelid)) == nrow(prop2016)
[1] TRUE
```

⁷ And, indeed, they are all unique.

1 Consider how we could use `match()` to achieve the desired result:

```
> trainmerged = cbind(traintrans,
+                       prop2016[match(traintrans$parcelid,
+                                       prop2016$parcelid),])
```

Oops, we ended up with two columns of parcel IDs. First, double check that the matching worked, and then redo excluding the redundant column:

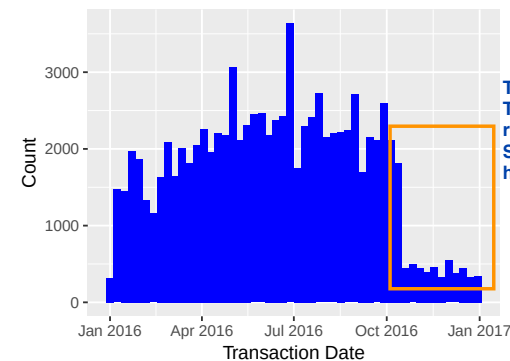
```
> all(trainmerged[,1]==trainmerged[,4])  
[1] TRUE
```

```
> trainmerged = cbind(traintrans,
+   prop2016[match(traintrans$parcelid,
+   prop2016$parcelid), -1])
```

1 EDA

2 First, we look at the transactions over time:

```
> ggplot(data=trainmerged, aes(x=transactiondate)) +  
+   geom_histogram(bins=52, fill="blue") +  
+   labs(x="Transaction Date", y="Count")
```



The missing chunk is test set
Test set is not chosen at random, it is chosen at later year
So you can make predictions by historical dataset

1 **Exercise:** Look at the output of

```
> summary(trainmerged)
```

2 Are there any concerns with any of the variables? Why would transforma-
3 tion help in some cases?

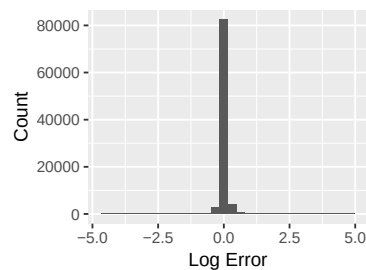
There are many missing values that we have to deal with

Some extreme values, including bedroomcount

- 1 Our main interest is in finding relationships that can help predict
- 2 `logerror`. Hence, it makes sense to construct plots that help to un-
- 3 cover possible useful predictors.
- 4 Visualization provides greater flexibility to discover nonlinear relation-
- 5 ships and interactions between variables. Formal models are crucial, but
- 6 they are limited by the assumptions that they make.

- 1 Start by taking a look at the distribution for `logerror`:

```
> ggplot(data=trainmerged, aes(x=logerror)) +
+   geom_histogram() + labs(x="Log Error", y="Count")
```



2

- 3 Most response values are close to zero:

```
> quantile(trainmerged$logerror, c(0.01, 0.1, 0.3, 0.7, 0.9, 0.99))
      1%      10%      30%      70%      90%      99%
-0.342500 -0.073600 -0.017100  0.030500  0.094400  0.463882
```

- 1 **Exercise:** What are implications of this distribution for `logerror`?

2 Decide what we want to focus on. One approach: focus on the extreme cases where large errors.

- 1 I choose to focus on separating the errors into large bins, grouping together
- 2 errors of similar magnitude:

```
> breaks = c(-Inf, -0.5, -0.05, -0.01, 0, 0.01, 0.05, 0.5, Inf)
> trainmerged$logerrorbin = cut(trainmerged$logerror, breaks,
+   labels=c(1:8), ordered_result=TRUE)
```

- 3 In what follows, we will explore the relationship between potential predic-
- 4 itors and this binned error.

- 1 We will start with an obvious candidate: The location of the property.
- 2 The package ggmap gives a simple interface for downloading maps from
- 3 Google Maps and incorporating the result into a figure created using
- 4 ggplot2.
- 5 The process starts by defining the latitude/longitude box for the plot. The
- 6 order is left, bottom, right, top.

```
> bbox = c(min(trainmerged$longitude),
+          min(trainmerged$latitude),
+          max(trainmerged$longitude),
+          max(trainmerged$latitude))/1000000
```

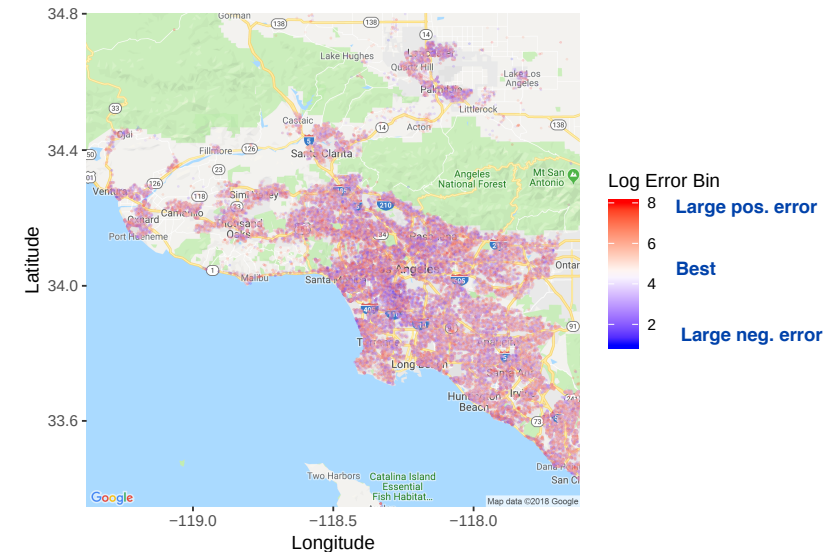
- 7 Note that the latitude and longitude were provided in microdegrees, and
- 8 need to be divided by 1000000 to convert to the (more typical) degrees.

- 1 The map is then create using the following syntax:

```
> googmap = get_map(bbox, maptype="roadmap")
```

- 2 We can then use this much as we did ggplot():

```
> ggmap(googmap) +
+   geom_point(data=trainmerged,
+             aes(x=longitude/1000000, y=latitude/1000000,
+               color=as.numeric(logerrorbin)),
+             alpha=0.15, size=0.05) +
+   labs(x="Longitude", y="Latitude", color="Log Error Bin") +
+   scale_color_gradient2(low="blue", high="red", mid="white",
+                         midpoint=4.5)
```

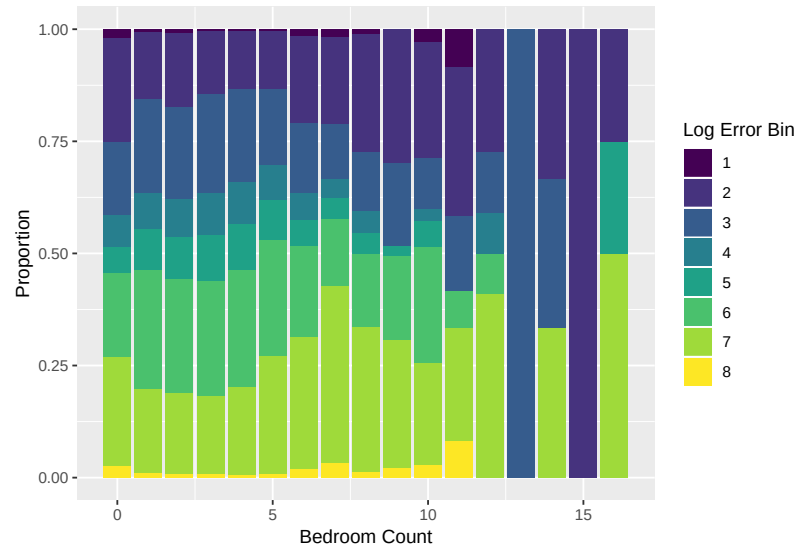
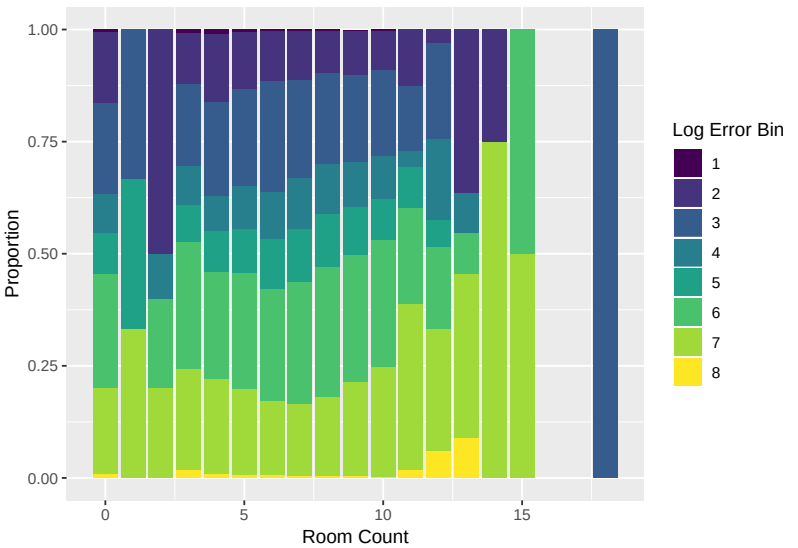


- 1 **Exercise:** How could we create a new plot which has one “dot” per zip
- 2 code, with color giving the median binned log error for that zip code?

1 When comparing the response with a categorical variable, or a continuous

2 variable with a finite number of possible values, a bar chart can be useful:

```
> ggplot(trainmerged, aes(x=bedroomcnt, fill=logerrorbin)) +  
+   geom_bar(position="fill") +  
+   labs(x = "Bedroom Count", y="Proportion",  
+       fill="Log Error Bin")  
>  
> ggplot(trainmerged, aes(x=roomcnt, fill=logerrorbin)) +  
+   geom_bar(position="fill") +  
+   labs(x = "Room Count", y="Proportion",  
+       fill="Log Error Bin")
```



1 **Exercise:** Interpret the plots on the previous pages.

2 _____

3 _____

4 _____

5 _____

6 _____

7 _____

8 _____

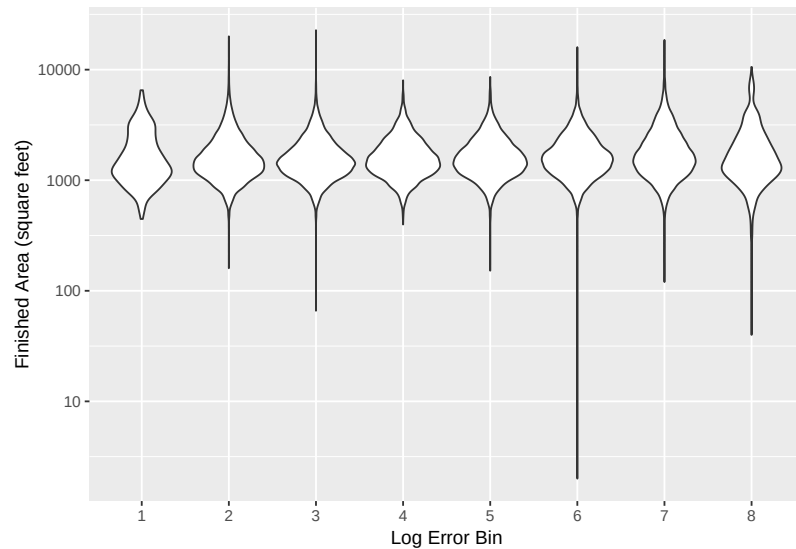
9 _____

10 _____

11 _____

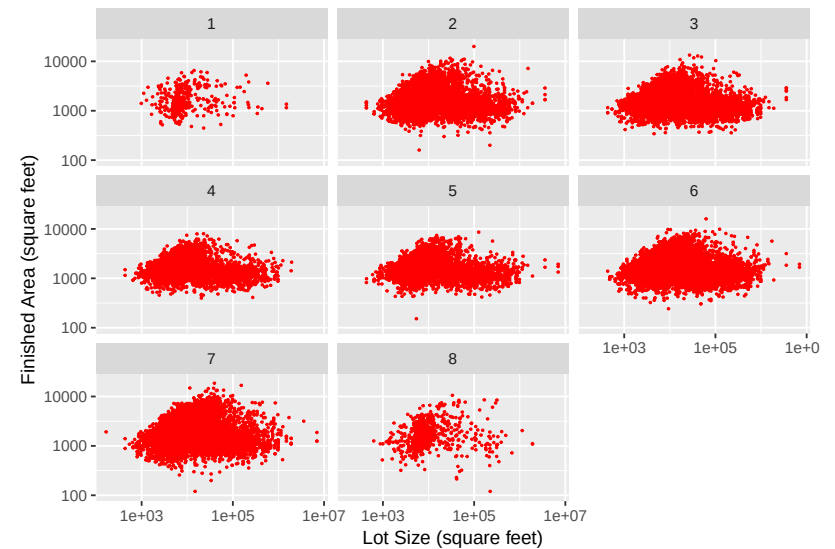
- 1 Violin plots are useful for comparing with a continuous predictor:

```
> ggplot(trainmerged, aes(x=logerrorbin,
+   y=calculatedfinishedsquarefeet)) +
+   geom_violin() + scale_y_log10() +
+   labs(x="Log Error Bin", y="Finished Area (square feet)")
Warning: Removed 661 rows containing non-finite values (stat_ydensity).
```



- 1 Testing for higher-order **interactions** can be fruitful, but this is where domain expertise starts to become very useful (or a great deal of patience), as
 2 there are many potential combinations to try.

```
> ggplot(trainmerged, aes(x=lotsizesquarefeet,
+   y=calculatedfinishedsquarefeet)) +
+   geom_point(size = 0.3, color="red") +
+   scale_x_log10() + scale_y_log10(limits=c(100,22000)) +
+   facet_wrap(~logerrorbin) +
+   labs(x="Lot Size (square feet)",
+   y="Finished Area (square feet)")
Warning: Removed 10487 rows containing missing values (geom_point).
```



Here is an example involving one categorical and one continuous variable:

```
> trainmerged$bathtub = cut(trainmerged$fullbathcnt,
+                             c(0,1,2,Inf))
> levels(trainmerged$bathtub) =
+   c("One Bathroom", "Two Bathrooms", "More than Two Bathrooms")
> ggplot(trainmerged, aes(x=factor(logerrorbin),
+                           y=yearbuilt)) +
+   geom_violin() +
+   facet_wrap(~bathtub) +
+   labs(x="Log Error Bin",
+        y="Year Built")
Warning: Removed 756 rows containing non-finite values (stat_ydensity).
```

